



— BIG DATA 72.80 · ITBA · 1ER CUATRIMESTRE 2026

Cloud Provider Analytics

De eventos crudos a respuestas de negocio, servidas desde Cassandra.

EQUIPO

Perez de Gracia, Mateo (63401) — Quian Blanco, Francisco (63006) — Stanfield, Theo (63403)

● Confiable · Con criterio · Sobre AstraDB real

Supongamos un proveedor de nube

Una plataforma cloud que factura a cientos de organizaciones cliente.

Qué pasa en la plataforma

- Las organizaciones consumen servicios: compute, storage, database, networking, analytics, genai.
- Cada acción deja un **evento de uso** con su costo (y, en la versión nueva, huella de carbono y tokens de GenAI).
- Alrededor hay **maestros** (organizaciones, usuarios, recursos), **tickets** de soporte y **facturación** mensual.



FinOps

Costos, consumo y anomalías por org / servicio.



Soporte

Tickets, SLA breach y satisfacción (CSAT).



Producto

Uso de servicios y adopción de GenAI.

Nuestro trabajo: convertir todo ese flujo crudo en analítica confiable para esas tres áreas.

Cinco preguntas que el negocio necesita responder

Q1 Costos y requests diarios por org y servicio en un rango de fechas.

FinOps

Q2 Top-N servicios por costo acumulado en los últimos 14 días.

FinOps

Q3 Evolución de tickets críticos y tasa de SLA breach por día.

Soporte

Q4 Revenue mensual con créditos/impuestos, normalizado a USD.

FinOps

Q5 Tokens GenAI y costo estimado por día.

Producto

*Todo el pipeline existe para responder esto — en tiempo casi real y desde Cassandra. **Al final volvemos a estas preguntas, ya respondidas.***

Por qué no alcanza con un SELECT sobre el crudo

Lo que complica la fuente

- Formato **EAV** (metric + value + unit): hay que pivotear a columnas por servicio.
- Dos esquemas conviviendo (v1 → v2): carbon_kg y genai_tokens aparecen a mitad del histórico.
- Nulos, unidades faltantes y costos negativos.
- Facturación **multi-moneda** que hay que llevar a USD.
- 43.200 eventos que llegan desordenados, en micro-lotes.

Las 5 Vs	En este proyecto
Volumen	43.200 eventos + 7 maestros.
Velocidad	Streaming en micro-lotes; watermark 60d.
Variedad	CSV + JSONL multi-schema (EAV).
Veracidad	Calidad + quarantine + dedupe.
Valor	5 marts query-first en Cassandra.

— LA ESTRATEGIA

Lambda + Data Lake por zonas, convergiendo en Cassandra



Lambda: el streaming cubre landing → Bronze de los eventos; maestros y facturación entran batch. Todo converge en Gold.

Ingesta con trazabilidad desde el día uno



Batch — maestros

- 7 maestros → Bronze Parquet particionado.
- Schema explícito (StructType, sin inferencia).
- Columnas técnicas: ingest_ts, source_file.
- dropDuplicates por clave natural.
- Partición por ingest_date.



Streaming — eventos

- `usage_events_stream/*.jsonl, trigger availableNow.`
- `withWatermark(timestamp, 60 días).`
- `dropDuplicatesWithinWatermark(event_id).`
- Checkpointing habilitado.
- Partición por date (fecha de evento).

Limpiar el dato antes de confiar en él

43.200

eventos ingeridos



41.222

válidos



1.978

en quarantine

- **Conformance v1/v2:** `coalesce(genai_tokens, 0); carbon_kg` queda null en v1 (no se fabrica).
- **Enriquecimiento:** broadcast join al maestro de orgs (`org_name, plan_tier, industry, hq_region`).
- **3 reglas de calidad:** R1 `event_id` nulo → quarantine · R2 costo negativo → se conserva + flag · R3 unit faltante con value → quarantine.
- **Anomalías multi-método:** z-score + MAD + p-tiles por servicio, con regla de consenso.

Consenso (≥ 2 métodos coinciden): de ~30% de eventos marcados a ~1,5% — una anomalía de verdad.

Modelamos por pregunta, no por tabla

Mart	Grano (PK)	Área	Colección	Filas
org_daily_usage_by_service	(org_id), event_date, service	FinOps	—	11.050
org_service_cost_14d	(org_id), total_cost_usd, service	FinOps · rollup	—	262
revenue_by_org_month	(org_id), month	FinOps	—	240
tickets_by_org_date	(org_id), event_date, severity	Soporte	—	984
genai_tokens_by_org_date	(org_id), event_date	Producto/GenAI	—	1.131
cost_anomaly_by_org_date	(org_id), anomaly_score, event_date, service	FinOps	set + map	598

Cada tabla se diseña para una consulta (query-first): sin joins ni full-scans en tiempo de lectura. Colecciones donde la evidencia es variable, escalares donde el conjunto es fijo.

Las cinco preguntas, respondidas en AstraDB

Pregunta	Tabla que la sirve	Técnica Cassandra
Q1 costo + requests por día	org_daily_usage_by_service	partición org + range-slice de fecha
Q2 top-N servicios (14d)	org_service_cost_14d	clustering DESC + LIMIT = top-N directo
Q3 tickets + SLA breach	tickets_by_org_date	clustering event_date DESC
Q4 revenue mensual en USD	revenue_by_org_month	normalización a USD calculada en Gold
Q5 tokens GenAI por día	genai_tokens_by_org_date	filtro service = 'genai'
+ top anomalías de costo	cost_anomaly_by_org_date	set methods + map scores

CQL en `cql/queries.cql`, ejecutadas en vivo sobre la CQL Console de AstraDB — capturas de resultados en el repo.

Por qué se puede confiar en estos números

Reprocesar no duplica

- Checkpointing en streaming (availableNow) + dedupe por event_id.
- Parquet en modo overwrite por dataset.
- Upsert por PK natural / truncate-reload en tablas con score en la PK.

Performance

- Particionado por fecha en Bronze/Silver/Gold.
- Lectura del snapshot más nuevo del maestro (evita duplicar el join).
- Sin sub-partición por servicio: evita archivos diminutos.

$\Delta = 0$ en las 6 tablas: la re-ejecución es idempotente end-to-end.

Tabla Cassandra	Run 1	Run 2
org_daily_usage_by_service	11.050	11.050
org_service_cost_14d	262	262
revenue_by_org_month	240	240
tickets_by_org_date	984	984
genai_tokens_by_org_date	1.131	1.131
cost_anomaly_by_org_date	598	598

Decisiones tomadas con criterio, no por defecto

Decisión	Por qué
Anomalías multi-método + consenso	Tres detectores votan; el valor está en la coincidencia, no en la unión (30% → 1,5%).
Colecciones donde modelan	set+map en anomalías (evidencia variable); escalares en el mart diario (conjunto fijo y tipado). No colecciones "decorativas".
Watermark de 60 días	Los eventos llegan desordenados sobre ~59 días de histórico; un watermark ajustado descartaría archivos válidos.
Python driver > conector Spark	El conector va atrás de Spark 4.0 / Scala 2.13.
Leer el snapshot más nuevo	Leer todas las particiones de un maestro duplicaba el join; se lee solo el último (bug encontrado y corregido).

El stack, y un flag para local o nube

Sp

PySpark 4.0

Structured Streaming + batch.

Pq

Parquet

Storage intermedio particionado por fecha, gestionado por Spark.

Ca

Cassandra 5 / AstraDB

Serving; desarrollo en Docker, evidencia final en AstraDB cloud.

</>

cassandra-driver (Python)

Carga de Gold — elegido sobre el conector Spark.

Jv

Java 21 · uv

Runtime de Spark + toolchain reproducible.

LOCAL + CLOUD

1 flag

SERVING_TARGET conmuta Docker local ↔ AstraDB cloud. El mismo `pipeline.py` corre en tu máquina (uv) o en Colab.

— CIERRE

Confiable, con criterio y sobre nube real

Lo que habilita el pipeline

- **FinOps:** costos y anomalías por org / servicio.
- **Soporte:** tickets críticos y SLA breach por día.
- **Producto:** adopción y costo de GenAI por org.

Próximos pasos (producción)

- Kafka real en vez de JSONL simulando micro-lotes.
- Full-streaming Silver → Gold → serving.
- BI directo sobre Cassandra (Grafana / Streamlit).

¡Gracias!